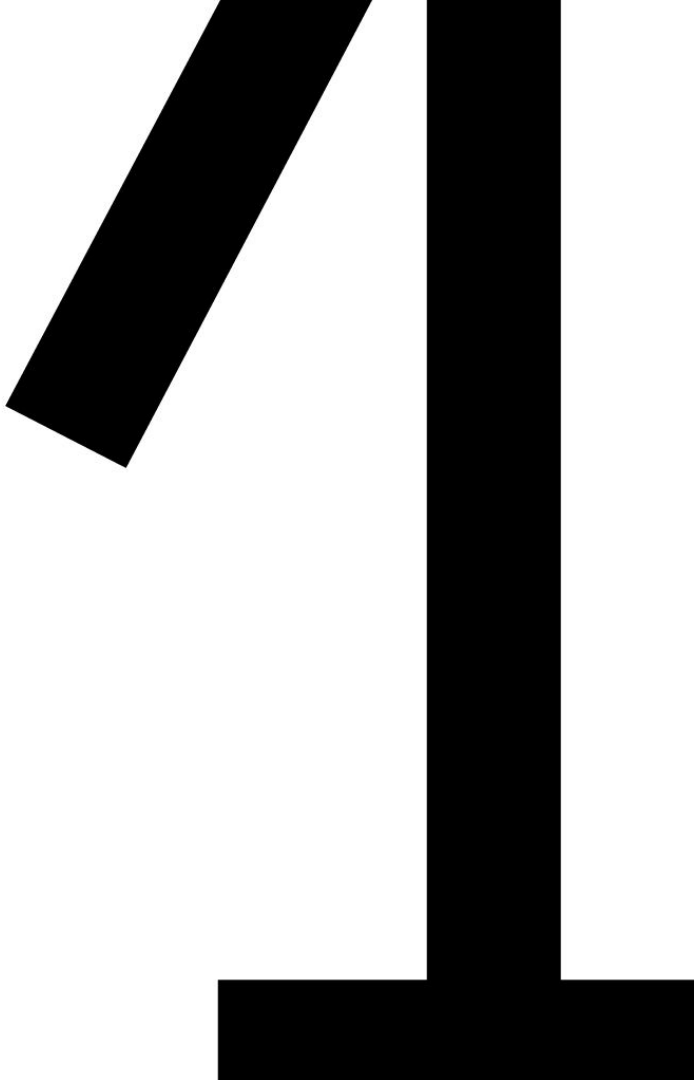
Sequence diagrams

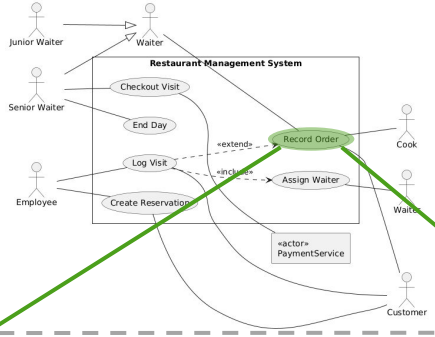
KdG Karel de Grote Hogeschool

What are we working on?



Three levels of behaviour of a system

Use case diagram



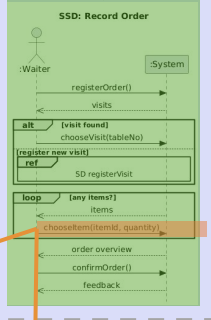
System level

Full description

Use case description

Use Case name	Record order
Goal	Register a new order
Actors	Waiter
Preconditions	All current visits are recorded
Postconditions	Order with order lines has been registered
Trigger event	Customer asks the waiter if he can order dishes
Main Success Scenario (MSS)	1. Waiter indicates he wants to take an order 2. System provides selection of current visits 3. Waiter indicates for which visit he will take the order 4. System registers new order 5. System shows choices of menu items 6. Waiter selects an item and enters quantity 7. System registers order line Repeat 5-7 until waiter stops ordering 8. System shows overview 9. Waiter confirms overview 10. System reports that order has been registered
Alternative scenarios	*a: incorrect entry 1. The complete order with any order line will be cancelled 2. Go to MSS Step 4 3a: visit not found 1. Go to UC register visit 7a: daily price deviates from standard price 1. Waiter also gives daily price in addition to quantity

SSD

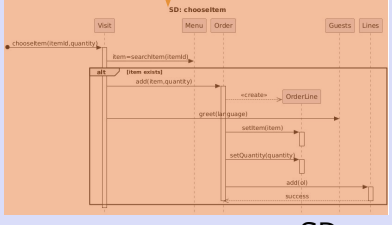


Only interactions

Use case level

Contract OC5 : chooseItem	
Operation:	chooseItem(itemId, quantity)
Cross-reference:	Use Case(s) : Record order
Precondition:	Instance o of Order was present
Postconditions:	Instance ol of OrderLine was created
	Instance ol of OrderLine was linked to instance o of Order Instance ol of OrderLine was linked to instance mi of MenuItem
	ol quantity got the input value

Operation contract



SD

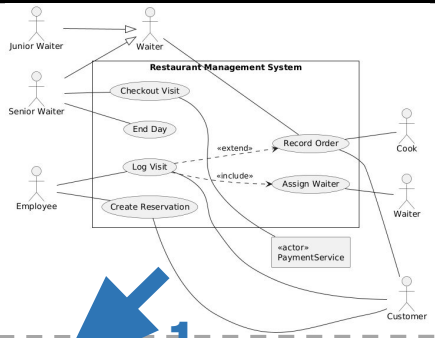
Operation/method level

Relationship SSD - OC - SD

- **For each Use Case, there is one System Sequence Diagram (SSD)**
- **For each System Sequence Diagram, there are multiple Operation Contracts**
  - Each input message in the SSD is one operation
- **For each Operation Contract, there is one Sequence Diagram (SD)**
  - The full operation of an operation is plotted in a Sequence Diagram
  - So, for each SSD, there are multiple Sequence Diagrams

Order of modelling

Use case diagram



System level

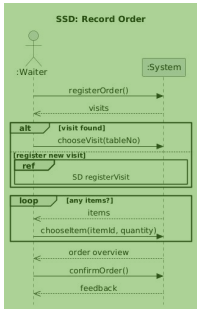
1

Use case description

Use Case name	Record order
Goal	Register a new order
Actors	Waiter
Preconditions	All current visits are recorded
Postconditions	Order with order lines has been registered
Trigger event	Customer asks the waiter if he can order dishes
Main Success Scenario (MSS)	1. Waiter indicates he wants to take an order 2. System provides selection of current visits 3. Waiter indicates for which visit he will take the order 4. System registers new order 5. System shows choices of menu items 6. Waiter selects an item and enters quantity 7. System registers order line Repeat 5-7 until waiter stops ordering 8. System shows overview 9. Waiter confirms overview 10. System reports that order has been registered
Alternative scenarios	*a: incorrect entry 1. The complete order with any order line will be cancelled 2. Go to MSS Step 4 3a: visit not found 1. Go to UC register visit 7a: daily price deviates from standard price 1. Waiter also gives daily price in addition to quantity

2

SSD

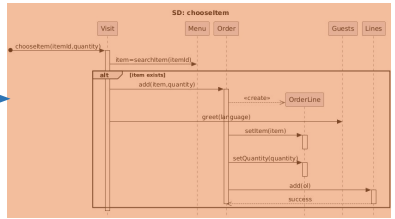


Use case level

3

Contract OC5 : chooseItem	
Operation:	chooseItem(itemId, quantity)
Cross-reference:	Use Case(s) : Record order
Precondition:	Instance o of Order was present
Postconditions:	Instance ol of OrderLine was created
	Instance ol of OrderLine was linked to instance o of Order Instance ol of OrderLine was linked to instance mi of MenuItem
	ol.quantity got the input value

4



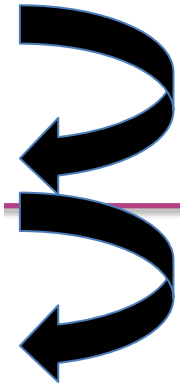
Operation/method level

Operation contract

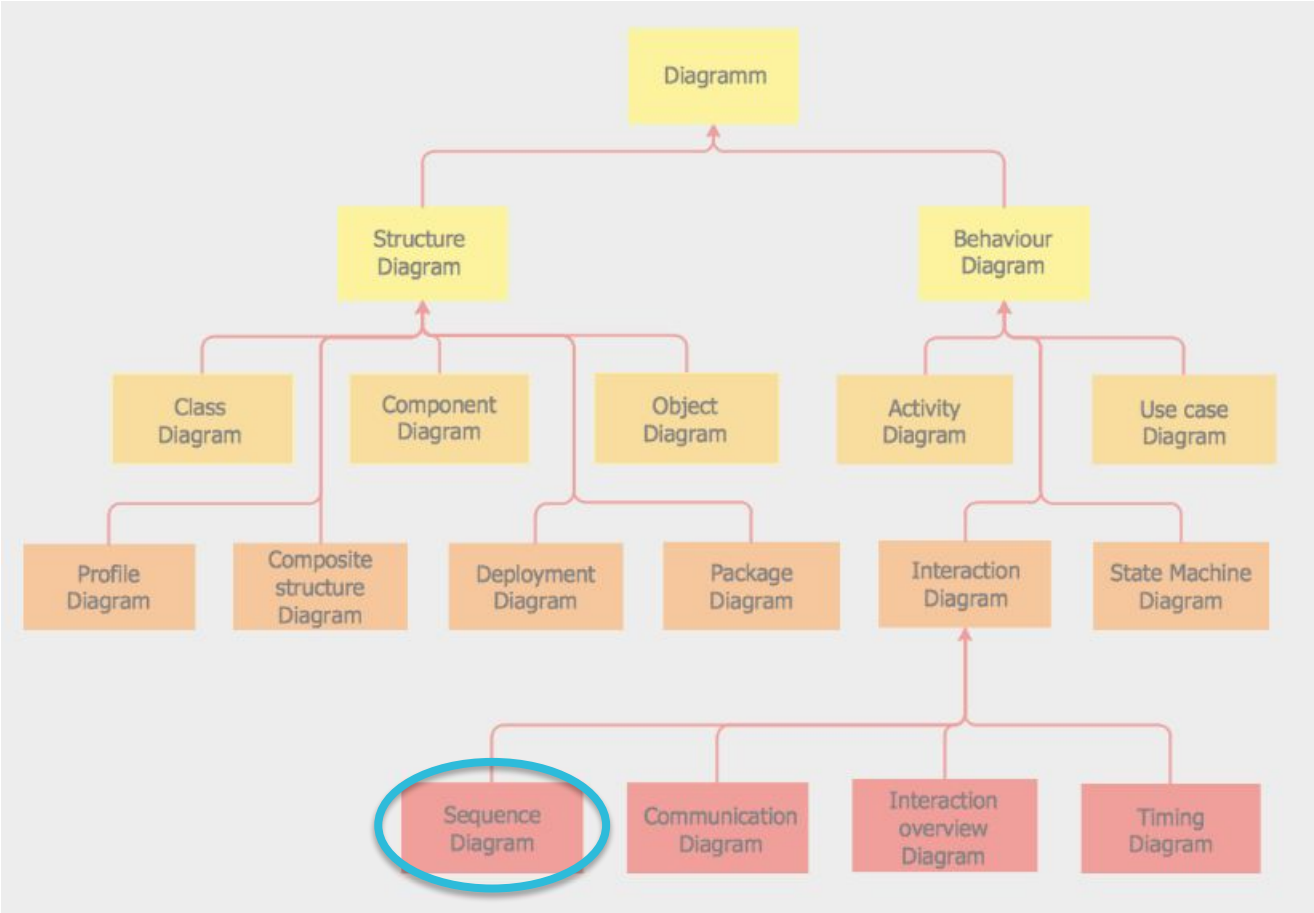
SD

Conceptual Thinking: Analysis -> Design

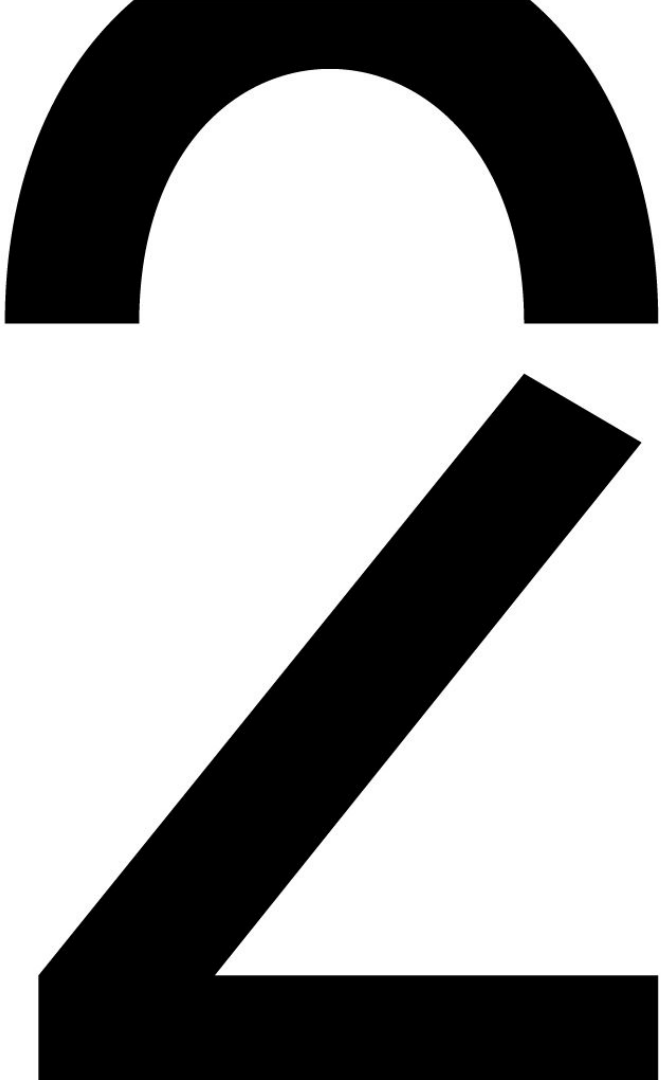
	STATIC / STRUCTURE	DYNAMIC / BEHAVIOUR / INTERACTION
ANALYSE	**DOMAIN MODEL** **= Conceptual class diagram**	**Use case diagram Use case descriptions** System Sequence Diagram Operation Contracts
DESIGN		**Sequence Diagram**



Overview of UML diagrams

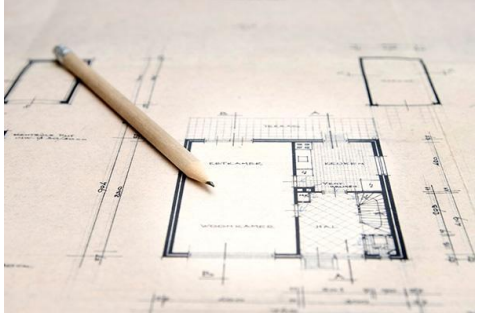


Introduction
Sequence Diagram



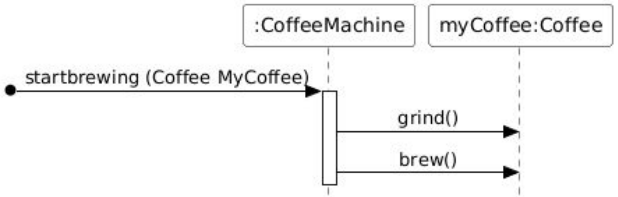
Interaction diagrams

- are mainly used in the design phase of a project: we look for the operations in the classes. Operations are called methods in implementation.
- show steps within an operation(method): which operations (methods) are called when executed on other objects?
- Input - from the analysis phase
  - *use cases*
  - *class diagrams*

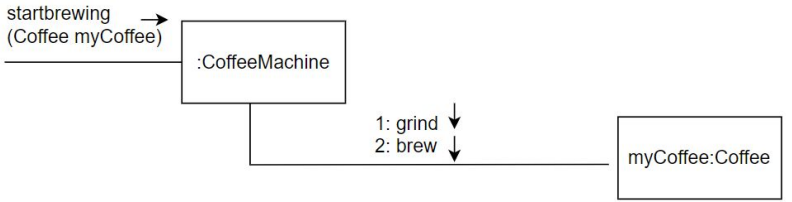


What are interaction diagrams

- Dynamic object models
- Important design tool
- Illustrate interaction between objects via messages
- 2 types : give +/- the same information
- Easy to read and convert to code

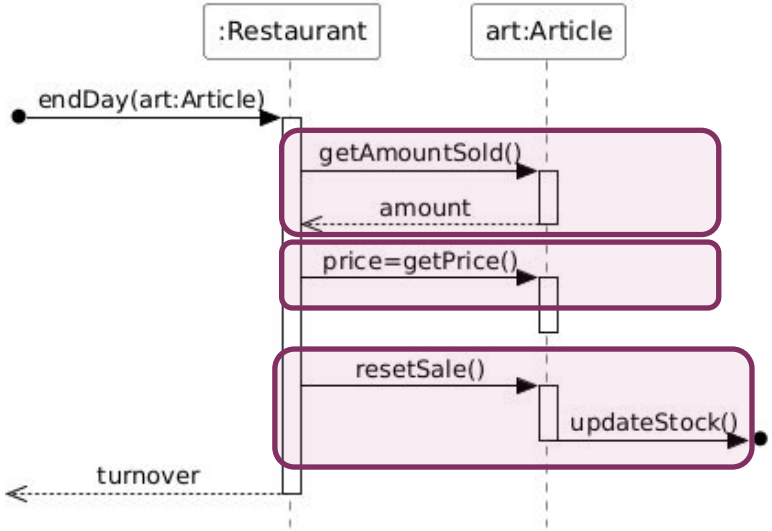


Sequence diagrams



Communication diagram

Example Sequence Diagram

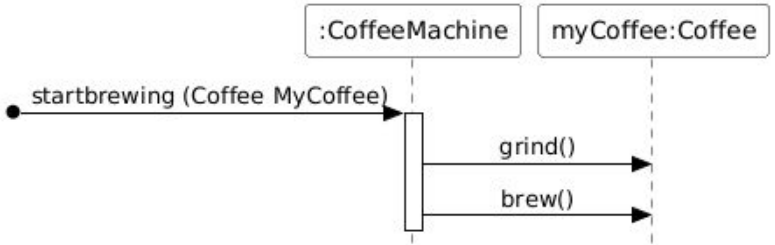


TIME

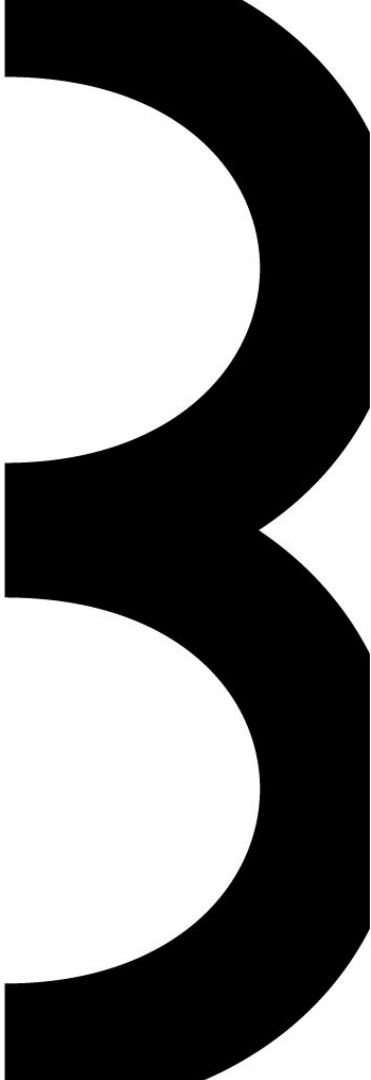
❑ interaction with emphasis on
- ⬇ the **time aspect**;
- ⟷ **interaction** between objects

Interaction converted to Java

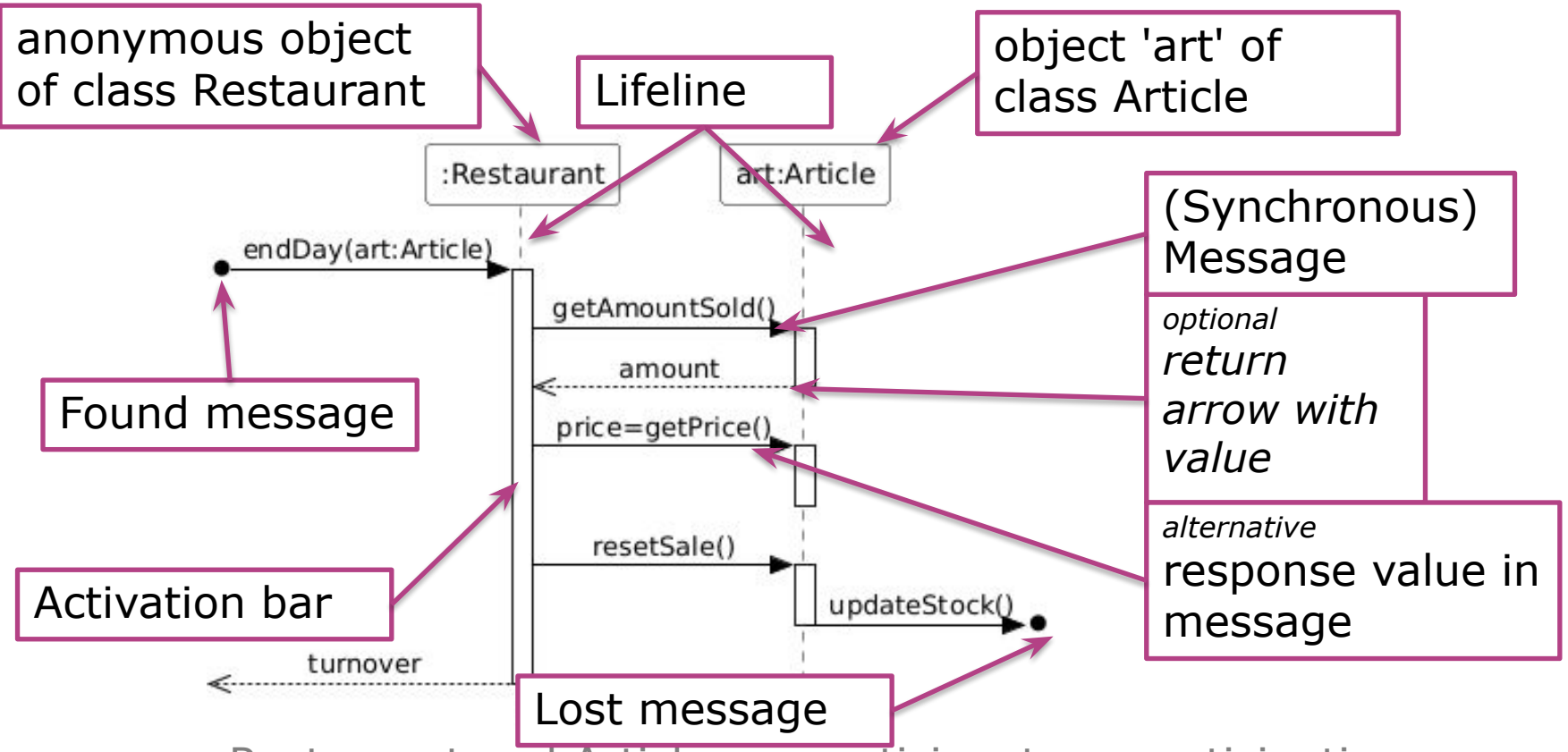
```
1  Public class CoffeeMachine{
2    public void startBrewing (Coffee
   myCoffee){
3      myCoffee.grind();
4      myCoffee.brew();
5    }
6  }
7
8  public class Coffee{
9    public void grind(){}
     public void brew(){}
10 }
```



Components of Sequence Diagram



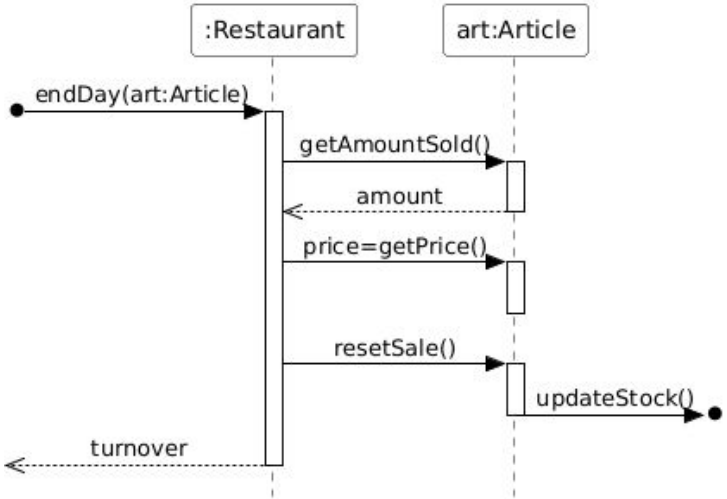
Concepts in a Sequence Diagram



Restaurant and Article are participants or participating properties

Basic elements

- **Objects are arranged at the top from left to right**
  - order is not fixed but arranged so that the diagram becomes most readable
- **Instances and lifeline:**
  - A dotted line is drawn down from each object: the lifeline

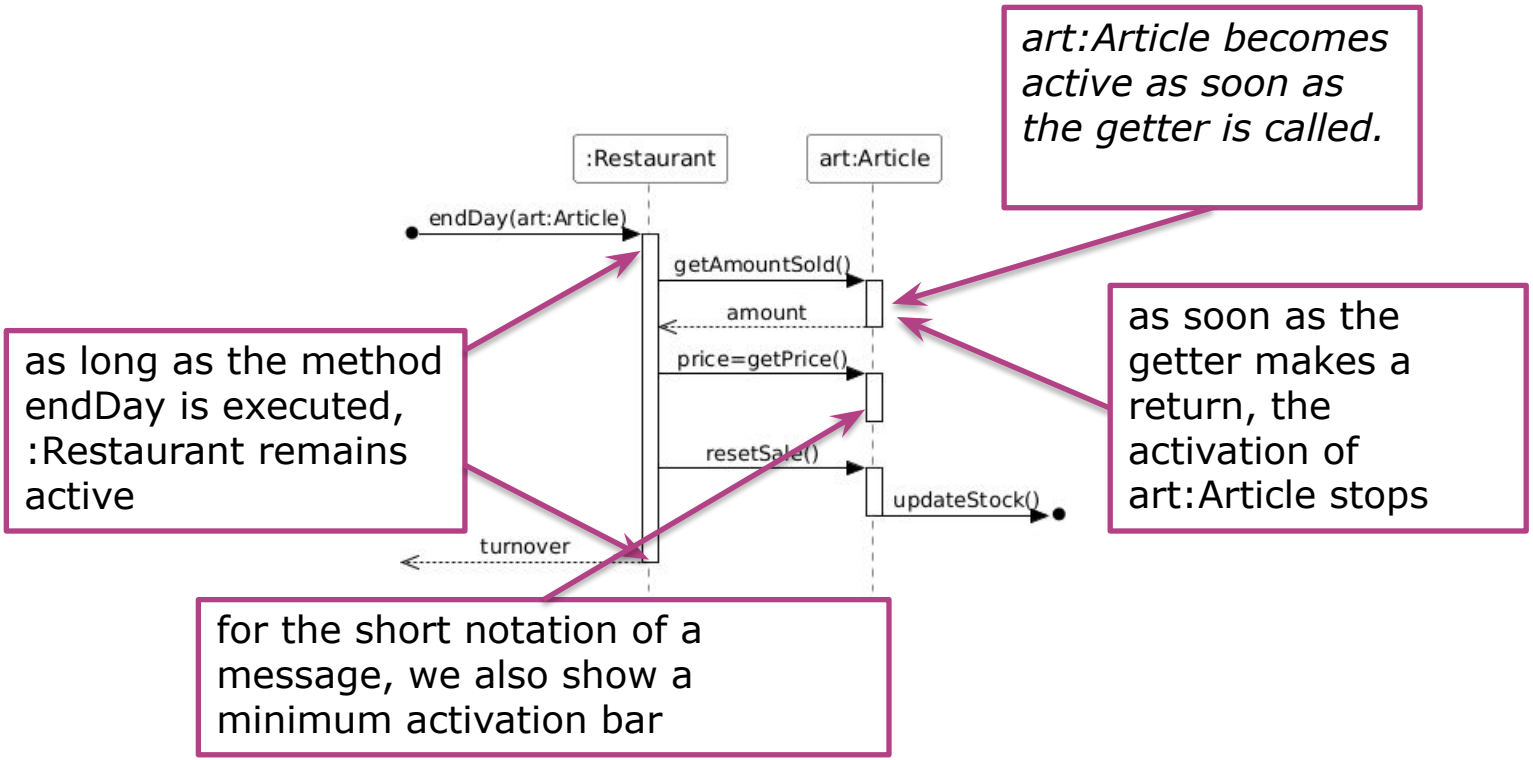


Concepts in a Sequence Diagram

- **Activation bar:** Indicates that the instance is active.
  (ie. during method execution)
  - **Length of bar** on a lifeline visually represents the duration of activation.
- **Interaction:** sequential steps within a method (white box).
  Often used in design of use case scenario.
- **Message:** each interaction step is a stimulus from one object to another object (message, method)

Time runs from top to bottom: a message at the top happens before a lower message.

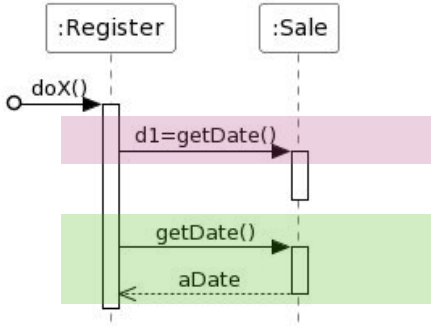
1. Activation bar : When is the object ACTIVE?



2. Messages : a) most standard usage

Synchronous message:
- solid line
- Ends with a FULL arrow
- *Optional answer message*
- name of message above the line

Reply message:
- dashed line
- ends with an OPEN arrow
- In reverse of the operation
- answer above the line



Reply message may be written in 2 ways

1.Short version : use syntax `returnVar = message(parameter).`
2.Standard version : as a separate reply message

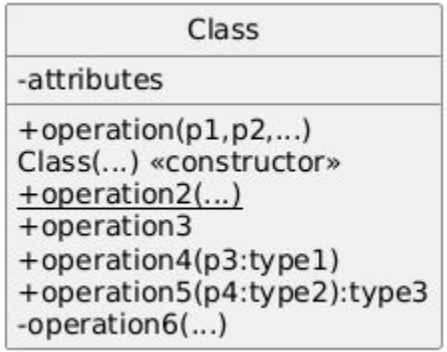
2. Messages : (b) syntax

Presumed knowledge

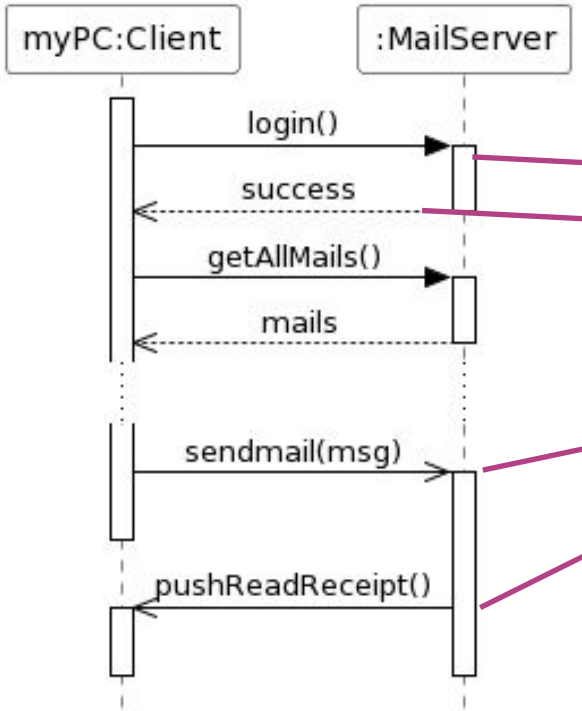
Form of message: return = message(parameter)

Example: d = getDescription(id)

Later we will see how these messages are incorporated into the class diagram as operations, but below is a sneak-preview already



2. Messages : (c) the asynchronous message



asynchronous message:

- **FULL line**
- Ends with an OPEN arrow

synchronous message

return value

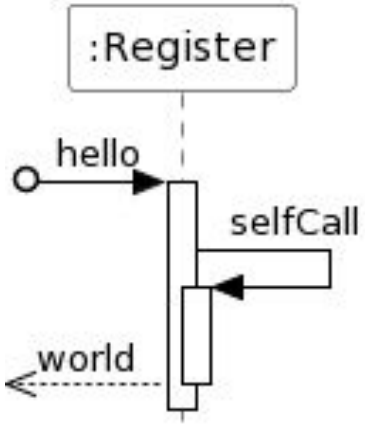
asynchronous

Caller does not have to wait for message to be processed: asynchronous

Asynchronous messages can not yield replies

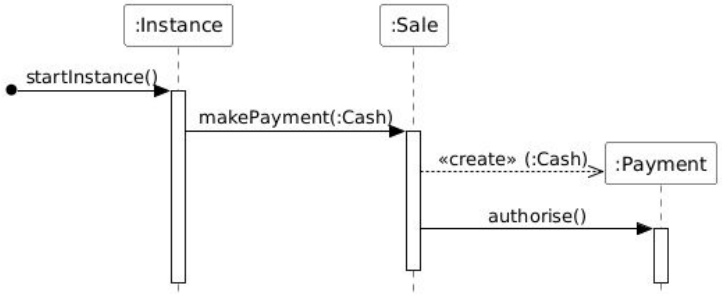
2. Messages : d) self / this message

Reflexive message (to this)



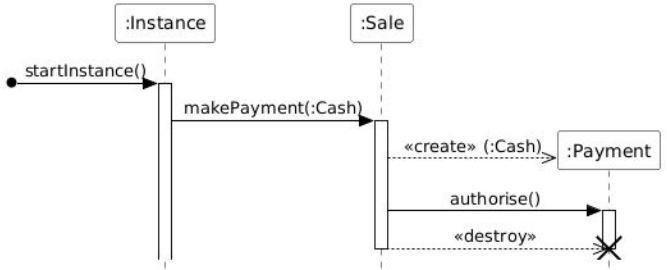
2. Messages : e) create & destroy

Creation message:
- Dashed line
- Message: <<create>> (arguments)
- Ends with an OPEN arrow
- Where arrow ends starts new object's lifeline

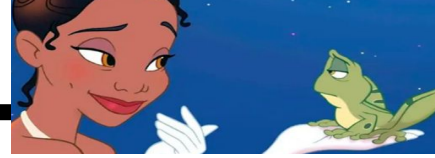


create

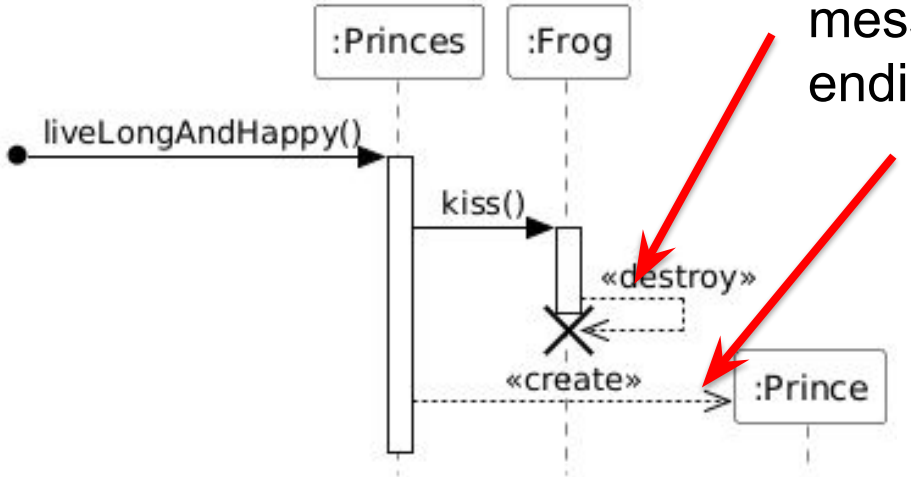
Destroy message:
- Dashed line
- Message: <<destroy>>
- Ends with an OPEN arrow
- Where arrow ends stops object's lifeline



destroy



2. Messages : e) create & destroy Example : A fairy tale



Object destruction: synchronous message to the object with a lifeline ending with X

Creation of object: "create"

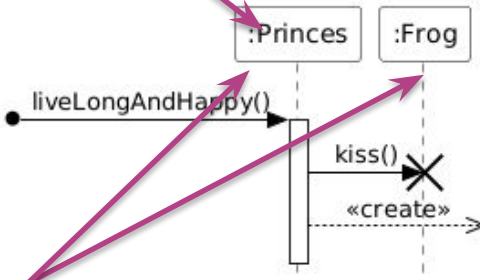
Discussie

Note "Creator" pattern :
- How are objects created?
- Who calls the constructor?

Creator

3. Lifeline

Lifeline box for an anonymous instance of class Princess



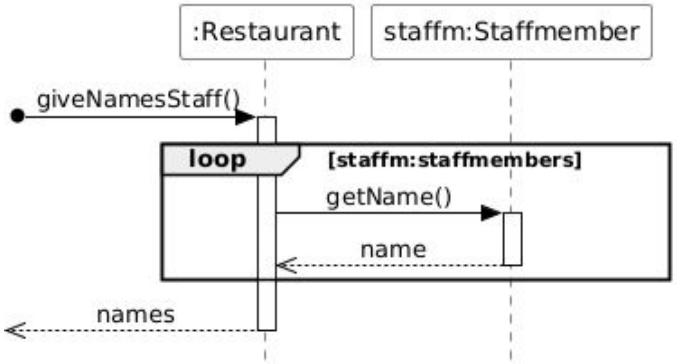
Lifeline box for an instance/object with name charming of class Prince

Princess and Frog were alive before SD started,
but Frog's lifeline ends after a 'destroy' (not possible in the tool used for this SD), the princess lives on quietly, as does the Prince

Lifeline starts below the box. Before it, this object did not exist.

4. Fragment: (b) Example

Sequence diagram
= Design ≠ Implementation
= Behavior or interactions ≠ Code



Guards are logical conditions that are checked in software before the getName() message is sent.

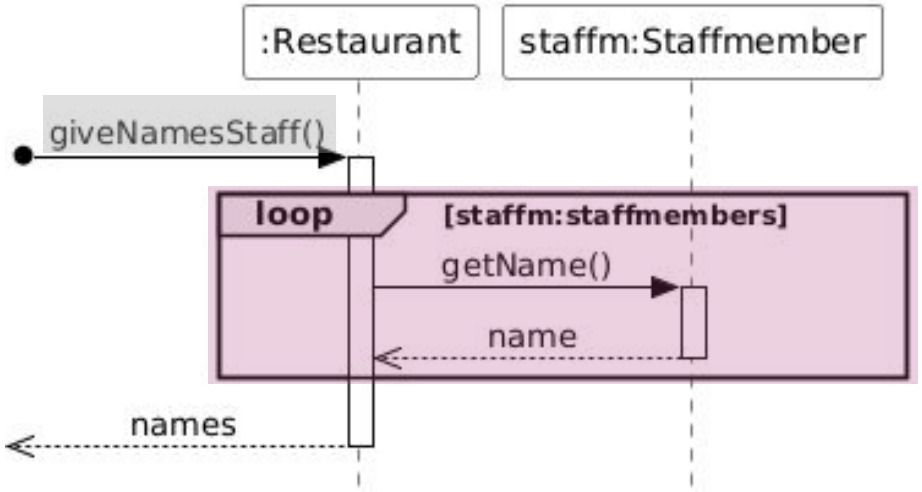
ATTENTION : The guard can only use information (attributes/arguments) that are available

4. Fragment : c) most commonly used

Frame Operator	Description
loop	**loop over the fragment as long as [guard] is TRUE**
alt	execute 1 of several alternative fragments according to the value of [guard]
opt	execute the optional fragment only when [guard] is true. This is special case of alt with only 1 alternative.
break	is always inside a loop fragment. when the guard condition is reached, the functionality in this fragment is executed first, after which the first outer loop is terminated
ref	refers/references to another sd frame
par	Run the given fragments in parallel
region/critical	designation of a 'critical region' where only 1 process can run

4. Fragment : loop

Everything within the fragment is repeated according to the [guard]: a condition in square brackets

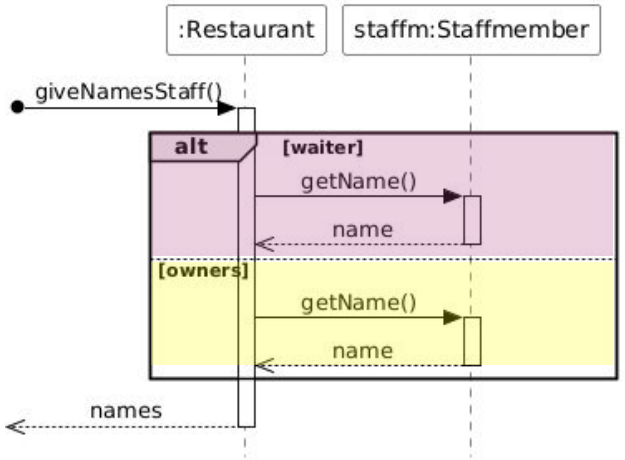


Meaning: Repeat the interaction as long as there's still a staffm object in staffmembers.

Result: The restaurant requests the name of each staff member (staffm) (getName()) and collects those names in a list (names).

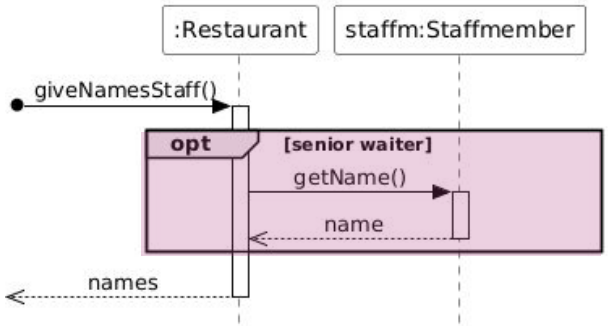
4. Fragment: alt

Depending on the value of [guard], 1 of the fragment's components is executed



4. Fragment: opt

Execute the fragment only if the condition within [guard] is met



4. Fragment : ref intro

- Sequence diagrams get big fast.

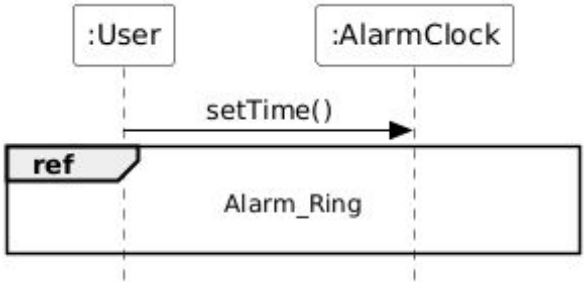
- The most important thing is to describe INTERACTION between objects.
- It is not the most ideal diagram to draw out the full logic

  activity diagrams they are better suited for this.

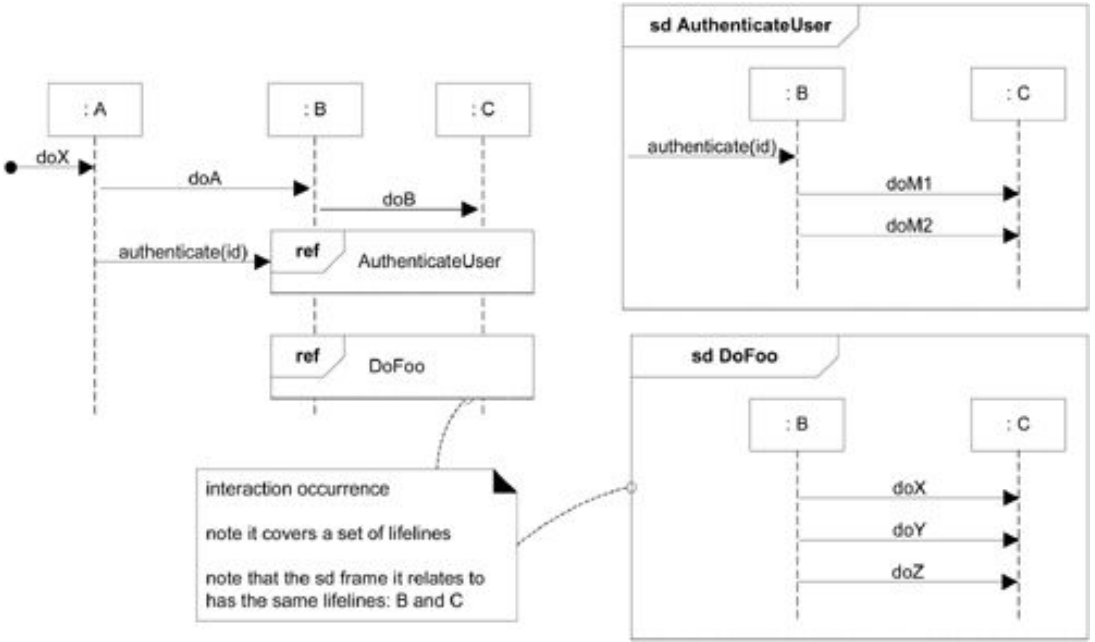
4. Fragment: ref

Readability is crucial!

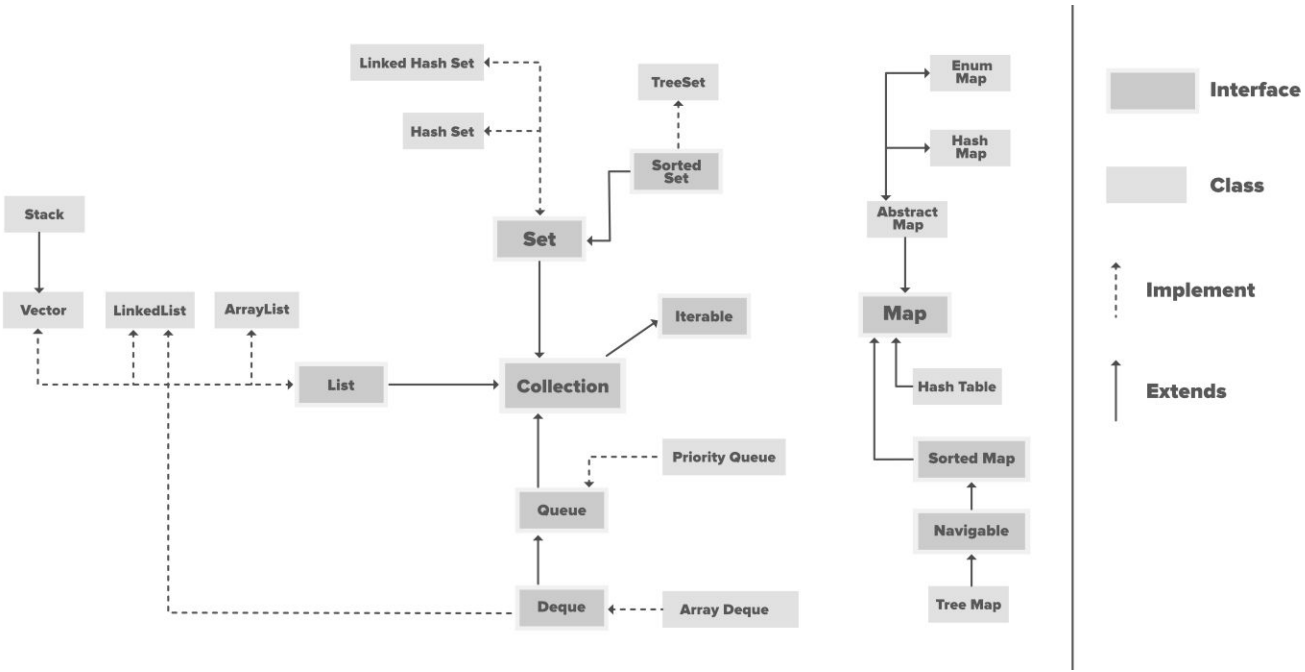
Use a ref fragment to elaborate on a part in a separate diagram.
For this, use the standard sd-frame notation where the name in the small box refers to another sequence diagram (sd)
Leave the rest of the frame blank.



4. Fragment : ref example

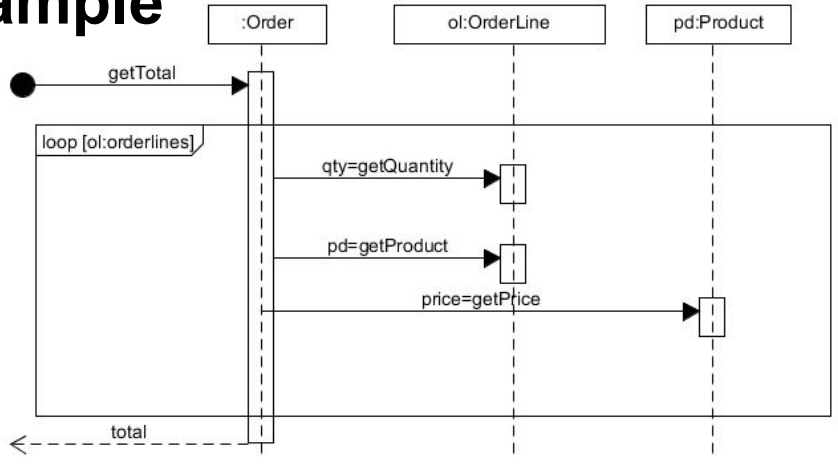


5. Collections : different types, similar interface



There are different types of Collection, each of which stores data via different data structures, yet can be filled and read in similar ways.

5. Collections : Other example




```
public double getTotal(){
  double total =0;
  for (OrderLine ol:orderlines) {
    int qty = ol.getQuantity();
    Product product = ol.getProduct();
    total = total + qty * product.getPrice();
  }
  return total;
}
```

Method of preparing Sequence Diagram

1. Determine which operation is the subject of the diagram
   - Indicate this operation on the found message arrow at the top left of the model
2. Determine what is the return value of the operation in the diagram
   - Draw this return value at the bottom left of the diagram
   - In the sequence diagram, we need to draw all interactions to obtain this return value
3. At the top of the SD, draw all objects used to obtain the return value
4. **Determine which operations should be called in which order**
   - You should be able to draw one continuous fictitious line from found message to the return value at the bottom left of the model
   - Operation = activation bar on a lifeline
   - Call of an operation/event = arrow between life lines
   - **For each call of an operation, define the parameters to be included**

Recurring Example: Monopoly

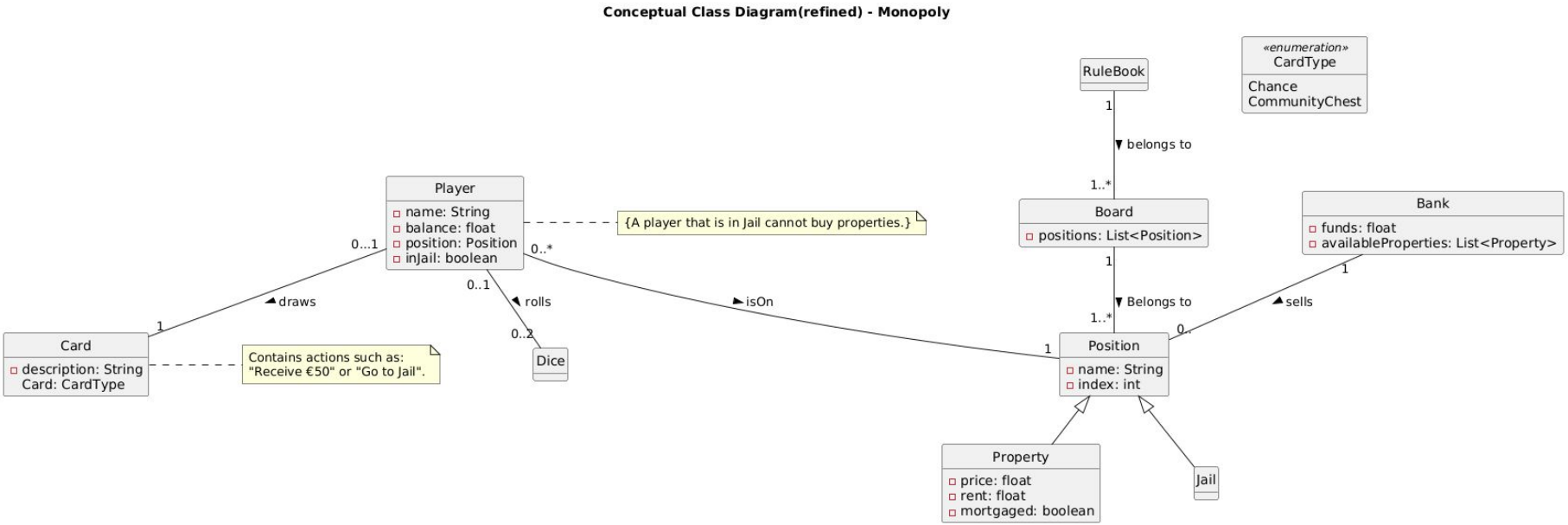
Purpose of the Exercise

This exercise forms an ongoing case in the Conceptual Thinking subject, and each week we apply w
digital version of the game Monopoly.

Why do we choose Monopoly?

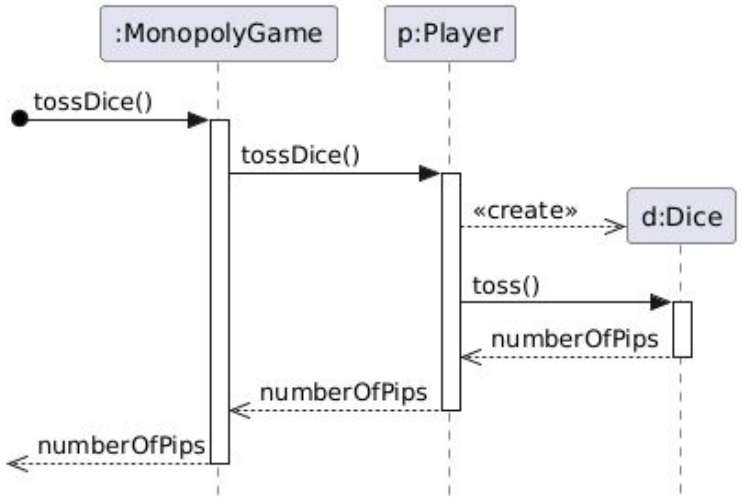
- Everyone knows it, or almost everyone
- It contains rich structures: players, money, rules, actions, boxes, cards
- It provides an ideal context for modelling object-oriented systems
- It is tangible and recognisable, making abstract concepts concrete

Recurring Example: Monopoly



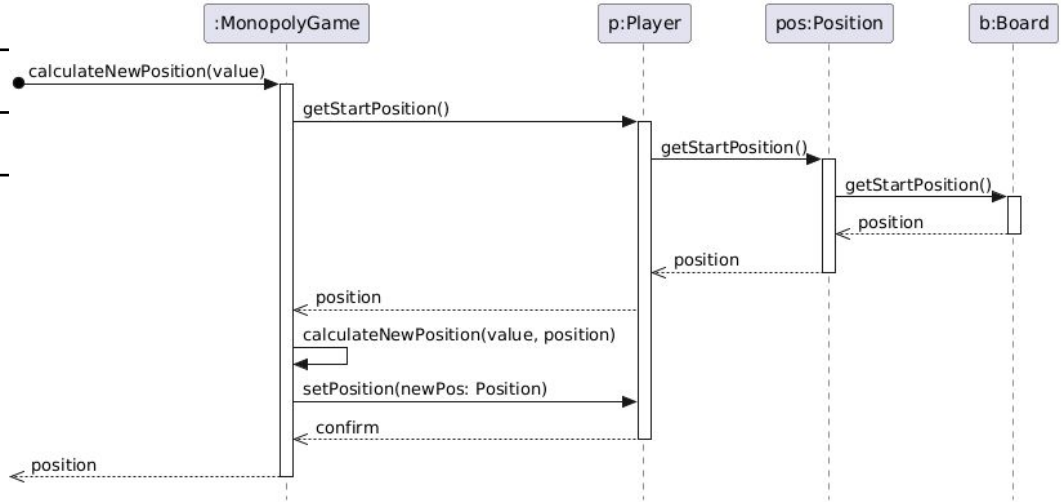
OC to SD

Contract OC1	
Operation	tossDice()
Cross-reference	Use case 1 - Toss dice
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists p and pos are linked
Postconditions	Created an instance d of the class Dice
	Link d to p
	N/A



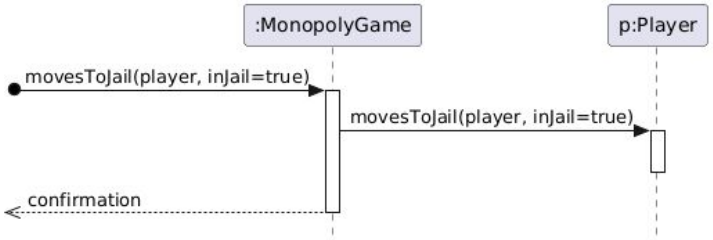
OC to SD

Contract OC2	
Operation	calculateNewPosition(value)
Cross-reference	Use case 1 - Toss dice
Preconditions	An instance b of the class Board exists An instance p of the class Player exists An instance pos of the class Position exists b and pos are linked p and pos are linked
Postconditions	N/A
	N/A
	Update position



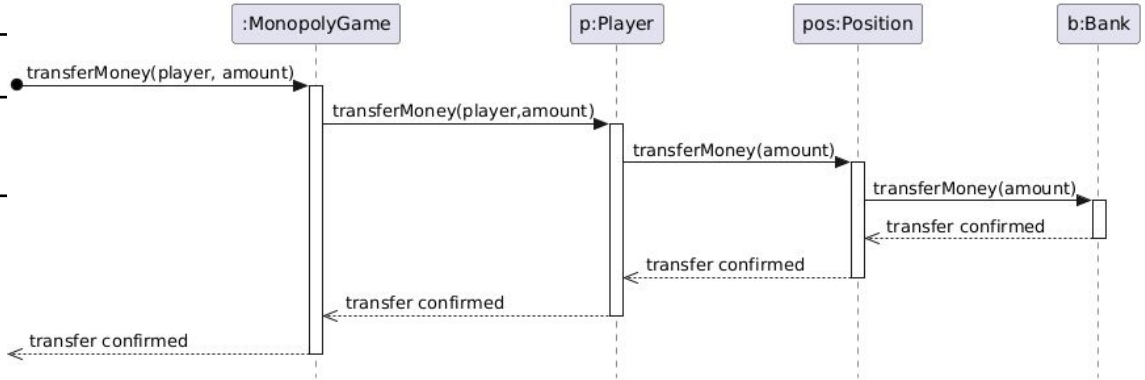
OC to SD

Contract OC3	
Operation	movesToJail(injail=true)
Cross-reference	Use case 1 - Toss dice
Preconditions	An instance b of the class Board exists An instance p of the class Player exists An instance pos of the class Position exists b and pos are linked p and pos are linked
Postconditions	N/A
	N/A
	Update injail



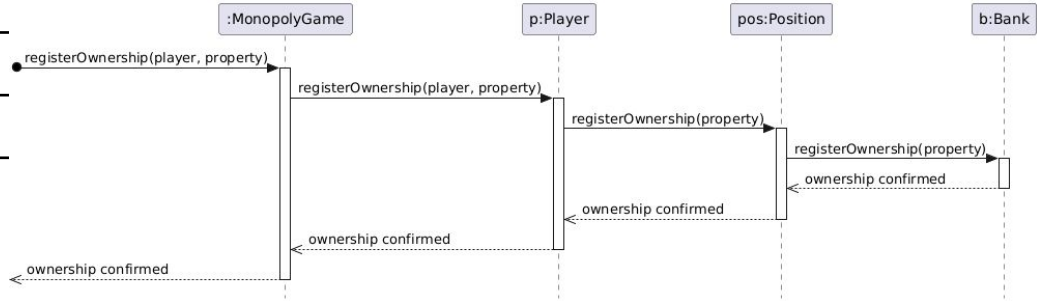
OC to SD

Contract OC4	
Operation	transferMoney(player, amount)
Cross-reference	Use case 2 - Buy property
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Postconditions	N/A
	N/A
	Update balance Update funds



OC to SD

Contract OC5	
Operatie	registerOwnership(player, property)
Cross-referenc e	Use case 2 - Buy property
Precondities	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Post conditions	N/A
	N/A
	Update availableProperties



OC to SD

Contract OC6	
Operation	evaluatePosition(newPosition)
Cross-reference	Use case 1 - Toss dice
Precondition	An instance b of the class Board exists An instance p of the class Player exists An instance pos of the class Position exists b and pos are linked p and pos are linked
Postconditions	N/A
	N/A
	N/A

OC to SD

Contract OC7	
Operation	detectFreeProperty()
Cross-reference	Use case 2 - Buy property
Precondition	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Postconditions	N/A
	N/A
	N/A

OC to SD

Contract OC8	
Operation	clickBuyProperty()
Cross-reference	Use case 2 - Buy property
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Postconditions	N/A
	N/A
	N/A

OC to SD

Contract OC9	
Operation	validateBalance(player)
Cross-reference	Use case 2 - Buy property
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Postconditions	N/A
	N/A
	N/A

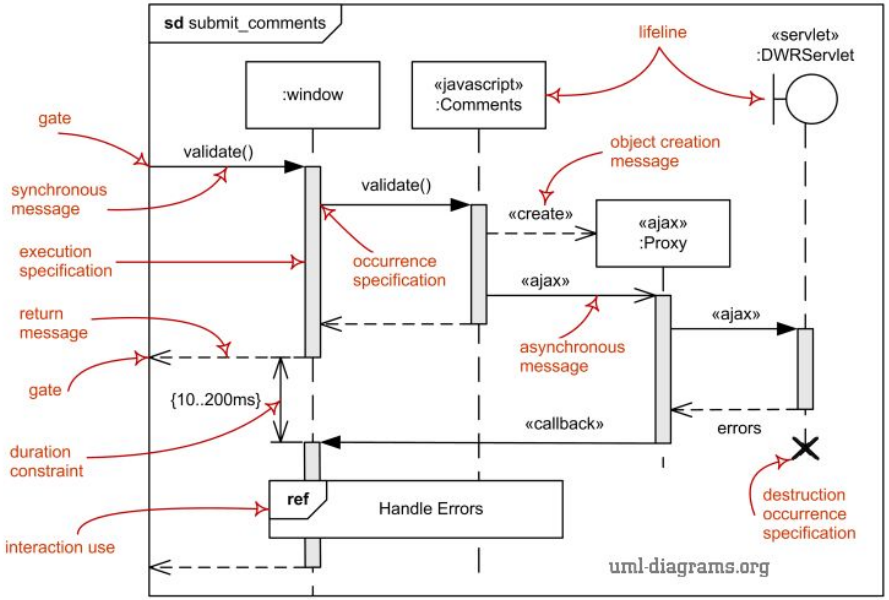
OC to SD

Contract OC10	
Operation	displayRules
Cross-reference	Use case 3 - Explain rules
Preconditions	An instance rb of the class RuleBook exists An instance b of the class Board exists rb and b are linked
Postconditions	N/A
	N/A
	N/A

Performance alternatives

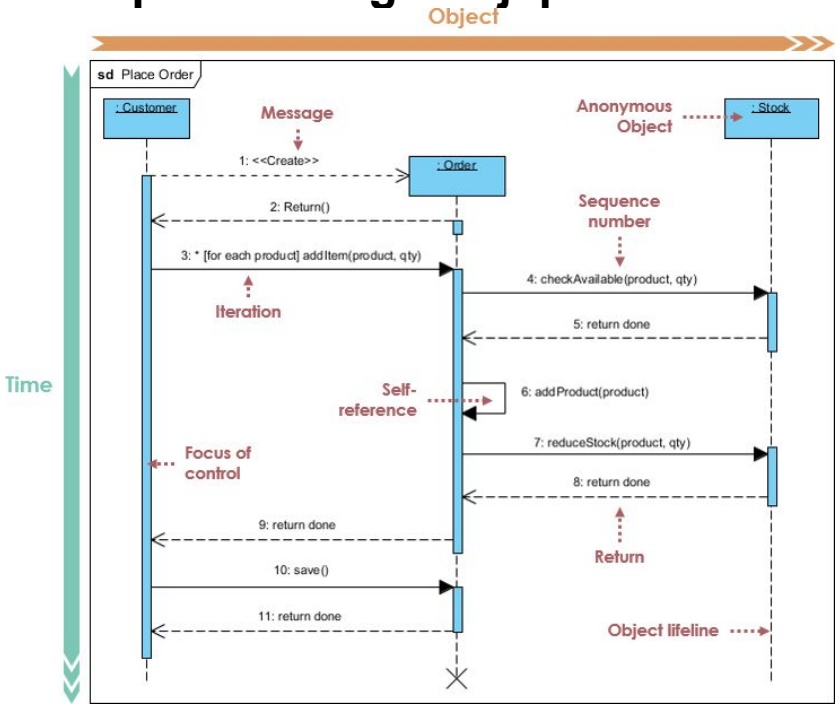
For information only

https://www.uml-diagrams.org/sequence-diagrams.html



Major elements of UML sequence diagram.

https://www.visual-paradigm.com/learning/handbooks/software-design-handbook/sequence-diagram.jsp



- p.

Questions?

KdG Karel de Grote Hogeschool